Known Issues

All known release issues related to this project will be documented in this file.

> *If you have a known issue that is not listed here, please open an issue on the project's GitHub repository.*

Control Center never becomes Ready when it wins the race against Kafka's DNS

Affects: `cp-enterprise-control-center-next-gen:2.5.0` on Minikube (`make cp-core-up`)

Symptom

`make c3-open` reports a healthy port-forward, but the browser shows `ERR_CONNECTION_REFUSED` on http://localhost:9021. `kubectl get pods -n confluent` shows the pod stuck at **2/3 Running with 0 restarts** — it never crash-loops and never recovers on its own:

```
NAME              READY   STATUS    RESTARTS   AGE
controlcenter-0   2/3     Running   0          45m
```

The readiness probe fails continuously:

```
Readiness probe failed: Get "http://10.244.0.7:9021/2.0/status/app_info":
connect: connection refused
```

Cause

All Confluent Platform pods are applied at once, so Control Center can start before the `kafka` headless Service has endpoints. C3 resolves `bootstrap.servers` once, eagerly, with no retry — when DNS is not ready yet, its `main` thread dies during Guice provisioning:

```
WARN  [main] Couldn't resolve server kafka:9071 from bootstrap.servers as
DNS resolution failed for kafka
Exception in thread "main" com.google.inject.ProvisionException:
  Caused by: KafkaException: Failed to create new KafkaAdminClient
  Caused by: ConfigException: No resolvable bootstrap urls given in
bootstrap.servers
```

Non-daemon Kafka Streams threads keep the JVM alive after `main` exits, so the container never terminates, the kubelet never restarts it, and it never binds port 9021. The pod stays in this state indefinitely.

Two details make this hard to spot:

- The **fatal exception is in the middle of the log, not at the tail** — `kubectl logs --tail` shows only `Unable to obtain lock as state directory is already locked by another process`, which is a downstream symptom of the dead `main` thread, not the cause. Grep for `No resolvable bootstrap urls` to confirm.
- `kubectl port-forward` attaches to any *Running* pod regardless of *readiness*, so the port-forward itself genuinely succeeds while nothing is listening on the other end.

Workaround

Restart the pod once Kafka is up; the StatefulSet recreates it and it reaches 3/3 in about a minute:

```
kubectl delete pod controlcenter-0 -n confluent
kubectl wait --for=condition=Ready pod/controlcenter-0 -n confluent --
timeout=240s
make c3-open
```

`make c3-open` is gated on `make c3-ready`, which waits for the pod to report Ready (up to `C3_READY_TIMEOUT`, default `180s`) and prints this diagnosis instead of opening a browser onto a dead port.